Proyecto Algoritmos y Estructuras de Datos

Lineamiento: *Línea 1, Gestión de Recursos Comunitarios Compartidos*

Integrantes: *Maria Fernanda Morales, Esteban Velasquez, Edward Castillo, Santiago Lopez*

Entrega 2: Informe actualizado (versión 2.0)

Control de versiones del informe

La versión 1.0 corresponde a la Entrega 1 y contenía el problema, los objetivos, el alcance, la metodología, 34 requerimientos funcionales y 34 historias de usuario distribuidos en 9 módulos (M01 a M09).

La versión 2.0 corresponde a esta entrega. Se reorganizan los módulos (quedan 8), se ajustan los requerimientos y las historias de usuario a lo implementado en el repositorio, se aplazan a la Entrega 3 los que dependen de grafos y árboles balanceados, se corrigen errores del informe anterior y se agrega la Fase 3 con la justificación de búsquedas y ordenamientos, la tabla Big-O, el diseño de la función hash con su análisis de colisiones y la medición de tiempos en Python. También se incorpora la integración y entrega continua (CI/CD) del repositorio con GitHub Actions.

En el texto se indica entre paréntesis el tipo de cambio: *(modificado)*, *(nuevo)*, *(reubicado)* cuando pasó a otro módulo, *(parcial)* cuando la lógica existe pero falta conectarla al menú o completarla, *(aplazado E3)* y *(corregido)*. El detalle de cada cambio está en la sección de trazabilidad.

Fase 1 (Analisis)

1. Problema (Necesidad)

1.1. Contexto del lineamiento

Gestión de Recursos Comunitarios Compartidos, el problema, en bibliotecas, salas de estudio, centros culturales y bancos de herramientas gestionan préstamos manualmente (cuadernos, hojas sueltas, acuerdos verbales), lo que genera reservas duplicadas, pérdida de trazabilidad, asignación injusta de turnos y ausencia de datos para decisiones de inversión, se necesita un sistema que administre inventario y usuarios, gestione préstamos/devoluciones con trazabilidad completa, ordene la demanda de forma justa y verificable, y genere información para sustentar decisiones.

1.2. Contexto del cliente

El cliente es la administración de una red de bibliotecas y salas de estudio universitarias, con varias sedes, cada una a cargo de un bibliotecario. Administran tres tipos de recursos, colección bibliográfica, espacios de estudio

y equipos tecnológicos. Su labor diaria incluye prestarlos, recibirlos, asignar turnos cuando hay demanda y darles mantenimiento o reposición.

Actualmente todo se maneja de forma manual y descentralizada (cuadernos, hojas de cálculo por sede, acuerdos verbales), sin visión unificada de la red. Esto genera problemas recurrentes: reservas dobles de un mismo espacio, equipos que se pierden sin rastro de quién los tiene, asignación de turnos poco transparente (no por orden de llegada), y falta de datos confiables para decidir compras, reparaciones o bajas al final del semestre.

Como consecuencia, hay pérdida de recursos, percepción de favoritismo, tiempo desperdiciado localizando objetos y decisiones de inversión mal informadas.

1.3. Contexto de la necesidad

La necesidad es tener un sistema centralizado, ordenado y verificable para gestionar recursos, usuarios y préstamos en toda la red. Surge porque la operación ya superó lo que el registro manual puede sostener, afectando sobre todo al bibliotecario (sin margen para corregir errores) y al administrador (que decide compras sin datos confiables).

El ciclo de préstamo es hoy ineficiente porque no se valida disponibilidad contra un registro común, no hay trazabilidad de quién tiene cada recurso, la fila de espera es informal, no se pueden deshacer errores de registro fácilmente, y localizar un recurso requiere revisar listados completos. La información clave a administrar incluye: inventario y estado de recursos, unidades disponibles por sede, datos de usuarios, préstamos y devoluciones con fechas, colas de espera, categorías del catálogo, sedes de la red e indicadores de uso.

No resolver esto mantiene la pérdida de recursos, la falta de equidad en los turnos y las decisiones de inversión sin sustento. La oportunidad es transformar ese registro disperso en información estructurada que soporte tanto la operación diaria como la toma de decisiones basada en evidencia.

1.4. Pregunta problema

¿Cómo se puede gestionar de forma centralizada, trazable y justa el préstamo de recursos en una red de bibliotecas y salas de estudio de un campus, evitando reservas duplicadas, sabiendo en todo momento quién tiene cada recurso y cuántas unidades quedan en cada sede, y asignando los turnos por estricto orden de llegada, y a la vez producir información confiable que permita decidir qué adquirir, reparar o dar de baja?

2. Objetivos

2.1. Objetivo general (modificado)

Diseñar e implementar, en Python, un sistema autónomo de gestión de recursos comunitarios para una red de bibliotecas y salas de estudio de campus, que administre el inventario de recursos y usuarios, gestione préstamos y

devoluciones con trazabilidad completa, ordene la demanda de forma justa y verificable, y genere información confiable para sustentar decisiones de inversión, utilizando estructuras de datos propias y algoritmos con eficiencia justificada y medida.

2.2. Objetivos específicos (modificado)

- Implementar el registro, consulta, actualización y baja de usuarios aplicando POO. La estructura de datos que utiliza es el vector dinámico propio. Corresponde al módulo M01.
- Modelar el catálogo de recursos con herencia (recurso base → libro, equipo, sala) y el registro de sedes, indexando los recursos por código. Las estructuras que utiliza son vector dinámico y tabla hash propia. Corresponde al módulo M02.
- Implementar el registro de préstamos y devoluciones contra el inventario de cada sede, conservando el historial y permitiendo deshacer la última operación. Las estructuras que utiliza son lista enlazada simple, lista doble, pila y tabla hash. Corresponde al módulo M03.
- Gestionar una lista de espera por recurso que respete estrictamente el orden de llegada y asigne la unidad automáticamente al devolverse. La estructura que utiliza es la cola circular. Corresponde al módulo M04.
- Organizar el catálogo en categorías sin códigos repetidos. Las estructuras que utiliza son árbol binario de búsqueda y tabla hash; la jerarquía arbitraria y el AVL quedan para la Entrega 3. Corresponde al módulo M05.
- Construir listados ordenados e informes de uso e inventario aplicando ordenamientos (1 básico + 2 avanzados) y búsqueda secuencial y binaria. Corresponde al módulo M06.
- Administrar la disponibilidad de cada recurso por sede. La estructura que utiliza es una matriz sede × recurso que crece automáticamente. Corresponde al módulo M07.
- Implementar funciones de valor agregado: recomendaciones, calificaciones, predicción de disponibilidad y gamificación. Corresponde al módulo M08.
- Modelar la red de sedes como grafo para encontrar la sede más cercana con el recurso disponible y detectar sedes aisladas (DFS/BFS). *(aplazado E3)*
- Garantizar que la funcionalidad produzca resultados correctos con pruebas de caso normal, límite y error, y medir empíricamente el rendimiento de los algoritmos. Es un requisito transversal que aplica a todos los módulos.

3. Alcance

3.1. Descripción de la solución (corregido)

Vamos a desarrollar un sistema de gestión de préstamos de consola, autónomo y sin conexión, que funciona como el "cerebro" del mostrador de cada biblioteca de la red y, al mismo tiempo, da una visión unificada de todas las sedes. No es una aplicación con interfaz gráfica; hasta este momento es un programa que el bibliotecario opera desde un menú de texto, manteniendo la información en memoria dentro de estructuras de datos construidas por el equipo. En la versión anterior se indicó que el sistema cargaba y guardaba en archivos planos; actualmente inicia con datos de ejemplo precargados y la persistencia en archivos planos queda para la Entrega 3.

3.2. Recursos

3.2.1. Recursos Tecnológicos

- *Equipos:* un computador por integrante (4 equipos en total), propiedad de cada integrante, aunque también serán facilitados por la universidad, para desarrollo individual y pruebas.
- *Lenguaje de programación:* Python 3.7 o superior, sin librerías externas, acorde al curso de Algoritmos y Estructuras de Datos.
- *IDE / Editor de código:* Visual Studio Code, vinculado al controlador de versiones.
- *Control de versiones:* GitHub, para el trabajo colaborativo y el control de cambios del proyecto entre los integrantes.
- *Integración y entrega continua (nuevo):* GitHub Actions, que ejecuta automáticamente las pruebas en cada cambio y genera la versión para entregar (ver 4.1).
- *Herramienta de gestión Kanban:* Notion, para implementar la metodología Kanban definida y mantener un flujo de trabajo estable (Backlog, Por hacer, En progreso, En revisión, Hecho).
- *Herramienta de diagramación UML:* Mermaid + StarUML, para los diagramas de clases.
- *Datos de prueba (modificado):* un conjunto pequeño precargado en `main.py` para la Fase 1 y, para esta entrega, conjuntos de 100 a 10.000 registros con el mismo formato de códigos del sistema (U001, R0001, P0001, S0001), usados para medir búsquedas, ordenamientos y la tabla hash.
- *Pruebas automáticas (nuevo):* `test_pruebas.py`, con 21 casos: tres por cada módulo de M01 a M07 (caso exitoso, caso borde y cuello de botella con 200 a 1.000 registros). Los resultados esperados están documentados en `PRUEBAS.md`.
- *Organización del repositorio (nuevo):* el repositorio de GitHub se divide en tres carpetas según las fases del ciclo de vida: `Analisis y planeación` (informes), `Diseño` (diagrama de clases y

modelos) y `Codificación y ejecución` (código, pruebas y guías de ejecución).

- *Documentación técnica:* se lleva en Notion y en COMO_EJECUTAR.md y PRUEBAS.md del repositorio, documentando cada estructura y función con su complejidad a medida que se desarrolla.

3.2.2. Recursos humanos

- *Equipo de desarrollo:* Maria Fernanda Morales, Esteban Velasquez, Edward Castillo y Santiago Lopez, encargados del diseño del modelo de datos, la implementación de los módulos, las pruebas y la documentación.
- *Docente:* Tatiana Cabrera, quien define los entregables por fase, evalúa los avances y guía metodológicamente al equipo.
- *Roles internos:* distribuir el trabajo de manera eficiente y justa entre los 4 integrantes, apropiándose cada uno de al menos un rol de trabajo donde se destaque su facilidad en este campo y así mismo poder otorgar un plazo de tareas y optimizar el tiempo de entrega.
- *Asignación de roles:* son roles rotativos entrega a entrega para tener un manejo justo de tiempo y trabajo para todo el equipo de desarrollo.

3.3. Tiempo

- En la fase de análisis se trabajará el problema, el alcance, los objetivos, la solución, los recursos y el Kanban. Los responsables son Edward y todo el equipo. Tiene una duración estimada de 3 días y no depende de ninguna fase previa.
- En la fase de planeación se desarrollarán los RQF, las HU, los criterios, los escenarios, la matriz y el diagrama de solución. Los responsables son Edward y María F. Tiene una duración estimada de 3 días y depende de la fase 1.
- En la fase de diseño se definirán los módulos, las estructuras justificadas, el diagrama de clases y el modelo de datos. Los responsables son María F. y todo el equipo. Tiene una duración estimada de 3 días y depende de la fase 2.
- En la fase de desarrollo se construirá el modelo de clases junto con las estructuras lineales. Los responsables son Esteban, Santiago y María F. Tiene una duración estimada de 5 días y depende de la fase 3.
- En la fase de análisis de algoritmos *(nuevo)* se implementarán búsquedas, ordenamientos y la tabla hash propia, y se elaborará el análisis Big-O y la medición de tiempos. Los responsables son Santiago y Esteban. Tiene una duración estimada de 4 días y depende de la fase 4.

- En la fase de pruebas se ejecutarán los casos normales, límite y de error, que además corren automáticamente en cada cambio mediante CI. Los responsables son Santiago y Esteban. Tiene una duración estimada de 2 días y depende de las fases 4 y 5.
- En la fase de documentación se elaborarán las guías del repositorio (COMO_EJECUTAR y PRUEBAS), el informe y la bitácora. Los responsables son Edward y Santiago. Tiene una duración estimada de 2 días y depende de las fases 5 y 6.
- En la fase de entrega y sustentación se subirá el proyecto a e-aulas y se preparará la exposición de 10 minutos. Los responsables son todos los integrantes del equipo. Tiene una duración estimada de 1 día y depende de las fases 1 a 7.

3.4. Módulos (modificado)

- *El módulo M01 (Gestión de usuarios),* tiene como propósito administrar el padrón de usuarios. Sus funciones principales son registrar, consultar por código, actualizar correo, dar de baja (baja lógica) y reactivar. Su usuario es Operario/Admin.
- *El módulo M02 (Recursos y sedes),* tiene como propósito administrar el catálogo y las sedes. Sus funciones principales son registrar libros, equipos y salas, consultar por código con tabla hash, dar de baja, listar por tipo y registrar sedes. Su usuario es Operario/Admin.
- *El módulo M03 (Préstamos, devoluciones y deshacer),* tiene como propósito ejecutar el ciclo del préstamo contra el inventario de cada sede. Sus funciones principales son registrar préstamo y devolución, listar activos, atrasados, por usuario y por sede, consultar por código y deshacer la última operación. Su usuario es Operario.
- *El módulo M04 (Turnos / lista de espera),* tiene como propósito asignar turnos justos por orden de llegada. Sus funciones principales son solicitar turno, ver el siguiente sin retirarlo, ver la fila, cancelar y asignar automáticamente al devolver. Su usuario es Operario.
- *El módulo M05 (Categorías del catálogo),* tiene como propósito organizar el catálogo en categorías con códigos únicos. Sus funciones principales son registrar y consultar categorías mediante árbol binario de búsqueda y tabla hash. Su usuario es Operario/Admin.
- *El módulo M06 (Informes y listados),* tiene como propósito producir informes y vistas ordenadas. Sus funciones principales son el reporte de uso, el reporte de inventario y los listados ordenados por nombre y por cantidad de préstamos. Su usuario es Admin.
- *El módulo M07 (Inventario),* tiene como propósito controlar las unidades de cada recurso en cada sede. Sus funciones principales son consultar, sumar, restar y fijar unidades y mostrar la matriz sede × recurso. Su usuario es Operario/Admin.

- *El módulo M08 (Sistema inteligente)*, tiene como propósito dar valor agregado a partir de los datos. Sus funciones principales son recomendaciones, calificaciones, predicción de disponibilidad y gamificación. Su usuario es Admin.

3.5. Precio (corregido)

3.5.1. Precio hipotético en el ambiente estudiantil

- *Software:* costo de $0, ya que todas las herramientas que se proponen (Git, GitHub, VS Code, Notion, Mermaid, StarUML) cuentan con versiones gratuitas y educativas suficientes para desarrollar en su totalidad el proyecto.
- *Infraestructura de cómputo:* costo de $0, pues se usan los equipos propios de los integrantes y no se requiere servidor ni nube, el sistema es autónomo y funciona en local.
- *Imprevistos:* costo mínimo o nulo, pues es un proyecto universitario sin una producción y distribución del producto, por lo que el riesgo financiero es prácticamente nulo.
- *Total:* tiene un costo nulo.

3.5.2. Precio real en el ambiente profesional

- *Mano de obra:* 360 h × $35.000/h (desarrollador junior) = $12.600.000, y 360 h × $60.000/h (desarrollador semi-senior) = $21.600.000.
- *Infraestructura de cómputo:* costo de $0, aplicaciones locales, no es necesario el uso de la nube.
- *Herramientas y licencias:* costo aproximado de máximo $500.000, aunque la mayoría son gratuitas.
- *Total:* entre $12.600.000 y $22.100.000, dependiendo del nivel de los desarrolladores.

4. Metodología

El equipo trabaja en dos etapas, primero el diseño conjunto y luego la implementación por entregas.

En la etapa de diseño, antes de escribir la primera línea de código, el equipo se reunió para acordar el modelo de datos, qué información describe a un usuario, un recurso, una sede, y cómo se relacionan entre sí. Este acuerdo se documentó en un diagrama de flujo compartido, para que cada integrante trabaje sobre él y el sistema funcione de manera coherente.

En la parte de implementación, el trabajo se organiza mediante los entregables definidos por la docente del curso.

- En la primera fase se construyó la funcionalidad esencial sobre un conjunto de datos pequeño, para validar que el diseño funcionara correctamente y corregir errores básicos antes de trabajar con mayor volumen.

- En la Entrega 2 *(modificado)* se trabajó con conjuntos de 500 a 10.000 registros para medir la búsqueda por código, los ordenamientos y la tabla hash. Las decisiones tomadas a partir de estas mediciones se documentan en la Fase 3.
- En esta entrega *(nuevo)* también se corrigieron errores detectados en el menú (préstamos que no descontaban inventario, la consulta del siguiente turno que lo retiraba de la fila, categorías repetidas que se sobrescribían, entre otros), y se agregaron pruebas por módulo en `test_pruebas.py`.

Cada estructura y función se documenta en el momento en que se escribe: qué hace, qué recibe, qué devuelve, su complejidad y por qué se eligió, para que el código sea legible por cualquier persona externa al proyecto.

4.1. Integración y entrega continua (CI/CD) (nuevo)

Para que cada cambio que sube un integrante no rompa lo que ya funciona, el repositorio de GitHub tiene un flujo de integración y entrega continua con GitHub Actions. Está definido en el archivo `ci-cd.yml`, ubicado en la carpeta `.github/workflows` de la raíz del repositorio, que es donde GitHub busca los flujos automáticos. Todos los pasos se ejecutan dentro de la carpeta `Codificación y ejecución`.

- *Integración continua (CI):* en cada push a la rama main y en cada pull request, GitHub Actions descarga el código y lo prueba en Ubuntu y Windows con Python 3.9, 3.11 y 3.13. En cada combinación verifica la sintaxis de todos los módulos (`compileall`), ejecuta los 21 casos de `test_pruebas.py` con pytest y arranca `main.py` para comprobar que el menú abre y cierra sin errores. Si algún caso falla, el flujo queda en rojo y el cambio no se integra hasta corregirlo.
- *Flujo de trabajo del equipo:* cada integrante trabaja en una rama propia por tarea del Kanban, abre un pull request hacia main y solo se fusiona cuando las pruebas pasan y otro integrante lo revisa; en ese momento la tarjeta pasa de "En revisión" a "Finalizado".
- *Entrega continua (CD):* cuando el equipo crea una etiqueta de versión para una entrega (por ejemplo `v2.0`), y solo si las pruebas pasan, el flujo empaqueta la carpeta `Codificación y ejecución` (código, pruebas y guías, sin archivos temporales) en un archivo .zip y lo publica como Release en GitHub. Ese .zip es el que se sube a e-aulas, de modo que lo entregado siempre es una versión probada.

Como es una aplicación de consola sin servidor, la entrega continua no despliega el sistema en un servidor: genera la versión lista para entregar y ejecutar con `python main.py`. Al configurar el flujo, la integración continua detectó que el caso exitoso de M04 en `test_pruebas.py` falla porque la prueba pide turno en la sede "Centro" sin haberla registrado; el error está en la prueba, no en el módulo (C11).

***Dale click para ver nuestro tablero* Kanban Board proyecto**

Fase 2 (Planeación)

1. Requerimientos funcionales (RQF)

Se conserva la numeración de la Entrega 1 para mantener la trazabilidad. Los requerimientos nuevos continúan desde el RQF-035 y los aplazados se listan al final.

1.1. Módulo M01 Gestión de usuarios

- *RQF-001* Registrar usuario *(modificado)*. Descripción: permite dar de alta un nuevo usuario capturando nombre y correo; el código se genera automáticamente (U001, U002...) y no se aceptan campos vacíos. Usuario: Operario/Admin.
- *RQF-002* Consultar usuario por código. Descripción: permite ubicar un usuario existente ingresando su código y visualizar sus datos, estado y préstamos activos. Usuario: Operario/Admin.
- *RQF-003* Actualizar datos de usuario *(modificado, parcial)*. Descripción: permite modificar el correo de un usuario ya registrado. Usuario: Operario/Admin.
- *RQF-004* Dar de baja usuario *(modificado, parcial)*. Descripción: permite inactivar un usuario sin eliminarlo, validando que no tenga préstamos activos; un usuario inactivo no puede recibir préstamos ni pedir turno. Usuario: Admin.
- *RQF-019* Buscar usuario por código en tiempo constante promedio *(reubicado)*. Descripción: el gestor de usuarios indexa cada usuario en una tabla hash propia; la búsqueda lineal de la Entrega 1 se conserva solo para comparar. Usuario: Operario/Admin.
- *RQF-035* Reactivar usuario *(nuevo, parcial)*. Descripción: permite volver a activar un usuario inactivo. Usuario: Admin.

1.2. Módulo M02 Recursos y sedes

- *RQF-005* Registrar recurso *(modificado)*. Descripción: permite dar de alta un libro (autor, ISBN, año), un equipo (tipo, modelo, ubicación) o una sala (capacidad, dotación) aplicando herencia desde la clase base Recurso; el código se genera automáticamente (R0001...). Usuario: Operario/Admin.
- *RQF-006* Consultar recurso por código. Descripción: permite ubicar un recurso mediante tabla hash y visualizar su tipo y estado. Usuario: Operario/Admin.
- *RQF-008* Dar de baja recurso *(parcial)*. Descripción: permite retirar un recurso dañado o para reposición; un recurso dado de baja no se puede prestar ni pedir en turno. Usuario: Admin.
- *RQF-020* Buscar recurso por código *(reubicado)*. Descripción: permite localizar un recurso en tiempo constante promedio mediante la tabla hash, ahora dentro de M02. Usuario: Operario/Admin.

- *RQF-023* Listar recursos por tipo *(modificado)*. Descripción: permite listar libros, equipos o salas por separado. Usuario: Admin.
- *RQF-042* Registrar y listar sedes *(nuevo)*. Descripción: permite registrar las sedes de la red; cada sede ocupa una fila de la matriz de inventario. Usuario: Admin.

1.3. Módulo M03 Préstamos, devoluciones y deshacer

- *RQF-011* Registrar préstamo *(modificado)*. Descripción: permite crear un préstamo validando que el usuario exista y esté activo, que el recurso exista y no esté dado de baja, que la sede exista y tenga unidades; descuenta una unidad y fija un plazo de 14 días. Si no hay unidades, ofrece pedir turno. Usuario: Operario.
- *RQF-012* Registrar devolución *(modificado)*. Descripción: permite cerrar un préstamo activo, devolver la unidad a la sede y asignarla automáticamente al siguiente en la fila si lo hay. Usuario: Operario.
- *RQF-013* Consultar préstamos de un usuario e historial *(modificado)*. Descripción: permite ver los préstamos activos de un usuario y el historial completo de préstamos, que se guarda en una lista doble y se puede recorrer del más antiguo al más reciente y al revés. Usuario: Operario/Admin.
- *RQF-015* Deshacer última operación *(modificado)*. Descripción: permite revertir el último préstamo o devolución usando una pila, restaurando el inventario y, si el préstamo salió de la fila, devolviendo al usuario al primer lugar. Usuario: Operario.
- *RQF-036* Listar préstamos activos *(nuevo)*. Usuario: Operario/Admin.
- *RQF-037* Listar préstamos atrasados con días de atraso *(nuevo)*. Usuario: Operario/Admin.
- *RQF-038* Consultar préstamo por código mediante tabla hash *(nuevo)*. Usuario: Operario.
- *RQF-039* Listar préstamos activos por sede *(nuevo)*. Usuario: Operario/Admin.

1.4. Módulo M04 Turnos / lista de espera

- *RQF-016* Registrar solicitud en cola de espera *(modificado)*. Descripción: permite encolar una solicitud solo cuando la sede no tiene unidades, validando usuario activo, recurso vigente y sede existente, sin permitir que un usuario esté dos veces en la misma fila. Usuario: Operario.
- *RQF-017* Asignar siguiente turno *(modificado)*. Descripción: al devolverse una unidad, el sistema la asigna automáticamente al siguiente en estricto orden de llegada, saltando usuarios inactivos. Usuario: Operario.
- *RQF-018* Consultar cola de espera de un recurso. Descripción: permite visualizar el orden completo de solicitudes pendientes y su cantidad.

Usuario: Operario.

- *RQF-040* Ver siguiente turno sin retirarlo de la fila *(nuevo)*. Usuario: Operario.
- *RQF-041* Cancelar solicitud *(nuevo, parcial)*. Usuario: Operario.

1.5. Módulo M05 Categorías del catálogo

- *RQF-025* Registrar categoría *(modificado)*. Descripción: permite dar de alta una categoría con código y nombre, rechazando códigos repetidos; se inserta en un árbol binario de búsqueda y en una tabla hash. Usuario: Operario/Admin.
- *RQF-026* Consultar categoría por código *(modificado)*. Descripción: permite ubicar una categoría por su código y conocer el total registrado. Usuario: Operario/Admin.

1.6. Módulo M06 Informes y listados

- *RQF-021* Listar recursos más prestados *(reubicado)*. Descripción: permite ordenar los recursos según su cantidad de préstamos con merge sort. Usuario: Admin.
- *RQF-024* Buscar recurso en listado ordenado *(reubicado, parcial)*. Descripción: permite aplicar búsqueda binaria (iterativa o recursiva) sobre un listado ordenado. Usuario: Admin.
- *RQF-034* Generar informe de inventario *(modificado)*. Descripción: permite ver cuántos libros, equipos y salas hay registrados. Usuario: Admin.
- *RQF-043* Generar reporte de uso *(nuevo)*. Descripción: permite ver el total de préstamos registrados. Usuario: Admin.
- *RQF-044* Listar recursos y usuarios ordenados por nombre con merge sort *(nuevo)*. Usuario: Admin.

1.7. Módulo M07 Inventario

- *RQF-009* Consultar disponibilidad por sede *(modificado, reubicado)*. Descripción: permite consultar las unidades disponibles de un recurso en una sede; el préstamo resta una unidad y la devolución la suma. Usuario: Operario/Admin.
- *RQF-010* Generar matriz de disponibilidad sede × recurso *(modificado)*. Descripción: permite visualizar las unidades de cada recurso en cada sede; la matriz agrega columnas automáticamente al registrar más recursos. Usuario: Admin.
- *RQF-045* Fijar unidades de un recurso en una sede *(nuevo)*. Usuario: Admin.

1.8. Módulo M08 Sistema inteligente

Requerimientos nuevos: la lógica está en la clase `SistemaInteligente` y se alimenta de los préstamos reales, porque el gestor de préstamos le informa cada préstamo, devolución y operación deshecha.

- *RQF-046* Recomendar recursos según usuarios con préstamos similares. Usuario: Admin.
- *RQF-047* Registrar calificaciones de 1 a 5 estrellas y generar ranking. Usuario: Admin.
- *RQF-048* Predecir la disponibilidad de un recurso según la duración promedio de sus préstamos. Usuario: Admin.
- *RQF-049* Gamificación con puntos, niveles, insignias y tabla de líderes. Usuario: Admin.

1.9. Requerimientos aplazados a la Entrega 3

RQF-007 Actualizar recurso, RQF-014 Historial de un recurso, RQF-022 Listar por antigüedad, RQF-027 Índice AVL de categorías, RQF-028 Sede más cercana con recurso disponible, RQF-029 Camino más corto entre sedes, RQF-030 Sedes aisladas, RQF-031 Informe de más prestados (se une con RQF-021), RQF-032 Tasa de devolución y RQF-033 Demanda por categoría.

2. Historia de usuario (HU)

Cada HU conserva el número de su RQF. Las aplazadas mantienen la redacción de la Entrega 1.

2.1. HU vigentes

- *HU-001 (RQF-001):* Como operario, quiero registrar un nuevo usuario con su nombre y correo y que el sistema le asigne un código, para poder habilitarlo a solicitar préstamos. *(modificado)*
- *HU-002 (RQF-002):* Como operario, quiero consultar los datos de un usuario por su código, para verificar su identidad antes de prestarle un recurso.
- *HU-003 (RQF-003):* Como operario, quiero actualizar el correo de un usuario, para mantener su información de contacto al día. *(modificado)*
- *HU-004 (RQF-004):* Como administrador, quiero inactivar a un usuario sin préstamos activos sin borrar su historial, para depurar el padrón. *(modificado)*
- *HU-019 (RQF-019):* Como operario, quiero buscar un usuario por su código de forma instantánea, para agilizar la atención en el mostrador.
- *HU-035 (RQF-035):* Como administrador, quiero reactivar un usuario inactivo, para que pueda volver a solicitar préstamos. *(nuevo)*
- *HU-005 (RQF-005):* Como operario, quiero registrar un nuevo recurso indicando su tipo (libro, equipo o sala) y sus datos propios, para incorporarlo al inventario. *(modificado)*
- *HU-006 (RQF-006):* Como operario, quiero consultar un recurso por su código, para conocer su estado antes de prestarlo.
- *HU-008 (RQF-008):* Como administrador, quiero dar de baja un recurso deteriorado, para que deje de ofrecerse en préstamo.

- *HU-020 (RQF-020):* Como operario, quiero buscar un recurso por su código de forma instantánea, para no revisar listados completos.
- *HU-023 (RQF-023):* Como administrador, quiero listar los recursos de un tipo, para revisar por separado libros, equipos y salas. *(modificado)*
- *HU-042 (RQF-042):* Como administrador, quiero registrar las sedes de la red, para llevar inventario y préstamos por sede. *(nuevo)*
- *HU-011 (RQF-011):* Como operario, quiero registrar un préstamo validando usuario, recurso y unidades de mi sede, para evitar reservas duplicadas. *(modificado)*
- *HU-012 (RQF-012):* Como operario, quiero registrar la devolución y que la unidad se asigne al siguiente en la fila, para liberar el inventario de forma justa. *(modificado)*
- *HU-013 (RQF-013):* Como operario, quiero ver los préstamos activos de un usuario, para resolver dudas sobre su actividad. *(modificado)*
- *HU-015 (RQF-015):* Como operario, quiero deshacer la última operación registrada por error con el inventario y la fila restaurados, para corregirla sin manipular datos a mano. *(modificado)*
- *HU-036 (RQF-036):* Como operario, quiero ver los préstamos activos, para saber qué recursos están fuera. *(nuevo)*
- *HU-037 (RQF-037):* Como operario, quiero ver los préstamos atrasados y sus días de atraso, para contactar a esos usuarios. *(nuevo)*
- *HU-038 (RQF-038):* Como operario, quiero consultar un préstamo por su código, para ver su sede, fecha y estado. *(nuevo)*
- *HU-039 (RQF-039):* Como administrador, quiero ver los préstamos activos de una sede, para conocer su carga. *(nuevo)*
- *HU-016 (RQF-016):* Como operario, quiero encolar la solicitud de un usuario cuando no hay unidades en la sede, para respetar su turno. *(modificado)*
- *HU-017 (RQF-017):* Como operario, quiero que el sistema asigne automáticamente el siguiente turno, para eliminar la percepción de favoritismo. *(modificado)*
- *HU-018 (RQF-018):* Como operario, quiero ver la cola de espera de un recurso, para informar a los usuarios cuántas personas hay antes que ellos.
- *HU-040 (RQF-040):* Como operario, quiero ver quién sigue en la fila sin retirarlo, para avisarle sin que pierda su turno. *(nuevo)*
- *HU-041 (RQF-041):* Como operario, quiero cancelar una solicitud, para que la fila avance. *(nuevo)*
- *HU-025 (RQF-025):* Como operario, quiero registrar categorías sin códigos repetidos, para mantener el catálogo organizado. *(modificado)*
- *HU-026 (RQF-026):* Como operario, quiero consultar una categoría por su código, para verificar a qué corresponde. *(modificado)*

- *HU-021 (RQF-021):* Como administrador, quiero ver un listado de los recursos más prestados, para priorizar su reposición.
- *HU-024 (RQF-024):* Como administrador, quiero buscar un recurso dentro de un listado ya ordenado, para encontrarlo de forma más eficiente.
- *HU-034 (RQF-034):* Como administrador, quiero un informe de cuántos recursos hay de cada tipo, para tener una visión general del catálogo. *(modificado)*
- *HU-043 (RQF-043):* Como administrador, quiero conocer el total de préstamos registrados, para medir el uso del sistema. *(nuevo)*
- *HU-044 (RQF-044):* Como administrador, quiero listados ordenados por nombre, para ubicar recursos y usuarios alfabéticamente. *(nuevo)*
- *HU-009 (RQF-009):* Como operario, quiero consultar cuántas unidades de un recurso hay disponibles en mi sede, para decidir si puedo prestarlo de inmediato.
- *HU-010 (RQF-010):* Como administrador, quiero ver una matriz de unidades por sede y recurso, para planear reposiciones. *(modificado)*
- *HU-045 (RQF-045):* Como administrador, quiero fijar las unidades de un recurso en una sede, para cargar o corregir el inventario. *(nuevo)*
- *HU-046 a HU-049 (RQF-046 a RQF-049):* Como administrador, quiero recomendaciones, calificaciones, predicción de disponibilidad y gamificación, para mejorar la experiencia del usuario y anticipar la demanda. *(nuevo)*

2.2. HU aplazadas a la Entrega 3

HU-007, HU-014, HU-022, HU-027, HU-028, HU-029, HU-030, HU-031, HU-032 y HU-033.

3. Diagrama de Clases

Diagrama de clases

Trazabilidad de cambios (Entrega 1 → Entrega 2)

1. **Módulos**
    - *M01 Gestión de usuarios:* se mantiene. Se agregó baja lógica y reactivación para no perder el historial.
    - *M02 Recursos e inventario:* se dividió en M02 Recursos y sedes y M07 Inventario, porque la matriz de disponibilidad requirió su propia clase (GestorInventario).
    - *M03 Préstamos y M04 Turnos:* se mantienen. El préstamo ahora se valida contra las unidades de cada sede y se integra con la fila.
    - *M05 Búsqueda por código:* se elimina como módulo. La búsqueda por código no es una funcionalidad independiente del usuario, sino una técnica que necesitan todos los gestores, por eso la tabla hash quedó dentro de M02, M03, M04 y M05.
    - *M06 Ordenamientos y M09 Informes:* se fusionan en M06 Informes y listados, porque los listados ordenados solo se usan para los informes del administrador.
    - *M07 Categorías:* pasa a ser M05. Se implementó el árbol binario de búsqueda sin balanceo; la jerarquía y el AVL quedan para la Entrega 3.
    - *M08 Red de sedes:* se aplaza a la Entrega 3, que es donde se ven grafos. La clase Grafo con BFS y DFS ya está en `estructuras/grafo.py`, sin integrar.
    - *Nuevos:* M07 Inventario (sale de M02) y M08 Sistema inteligente.

2. **Requerimientos funcionales**

    De los 34 RQF originales, 4 se mantienen, 20 se modificaron o reubicaron y 10 se aplazan a la Entrega 3. Se agregan 15 nuevos (RQF-035 a RQF-049). Las HU siguen los mismos cambios de su RQF.

    - *Se mantienen:* RQF-002, 006, 008 y 018.
    - *RQF-001:* el código pasa a ser automático, porque si lo digita el operario se puede repetir.
    - *RQF-003:* solo se actualiza el correo, porque el nombre identifica al usuario en el historial.
    - *RQF-004:* "eliminar o inactivar" pasa a baja lógica, porque eliminar rompe el historial.
    - *RQF-005:* cada tipo de recurso registra sus atributos propios y la categoría pasa a M05.
    - *RQF-009:* pasa a M07 y la matriz guarda una sola cantidad disponible por celda.
    - *RQF-010:* la matriz sede × día se reemplaza por sede × recurso, porque el sistema no maneja reservas por fecha y lo que necesita el bibliotecario es

saber cuántas unidades hay por sede.

- *RQF-011:* antes solo se permitía un préstamo por recurso en todo el sistema; ahora decide el inventario de cada sede y se ofrece turno si no hay unidades.
- *RQF-012 y RQF-017:* la devolución asigna automáticamente la unidad al siguiente en la fila.
- *RQF-013:* muestra los préstamos activos del usuario y el historial completo se consulta desde el menú recorriendo la lista doble en ambos sentidos.
- *RQF-015:* el deshacer ahora restaura el inventario y el turno.
- *RQF-016:* se agregaron validaciones y la opción de pedir turno desde el menú.
- *RQF-019 y RQF-020:* la búsqueda por código del antiguo M05 queda dentro de M01 (usuarios) y M02 (recursos), cada uno con su tabla hash.
- *RQF-021 y 024:* se reubican del antiguo M06 Ordenamientos al nuevo M06 Informes y listados.
- *RQF-023:* el listado pasa a ser por tipo, porque los recursos aún no tienen categoría asociada.
- *RQF-025 y RQF-026:* se registra y consulta por código sin repetidos; las subcategorías y el subárbol quedan para la Entrega 3.
- *RQF-034:* el informe muestra el conteo por tipo y el detalle por sede está en la matriz de M07.
- *Aplazados:* RQF-007, 014, 022 y 027 a 033.
- *Nuevos:* RQF-035 a RQF-049, surgidos al construir y probar el menú.

3. Correcciones

- *C1.* El objetivo general indicaba implementación en C++ y Python; el proyecto se desarrolla solo en Python.
- *C2.* Se indicaba persistencia en archivos planos; actualmente el sistema trabaja en memoria.
- *C3.* En el precio, la mano de obra no coincidía con 360 h × tarifa y había un texto incompleto. Se recalculó a $12,6 – $22,1 millones.
- *C4.* La matriz sede × día se reemplazó por sede × recurso.
- *C5.* La búsqueda de usuario recorría el vector completo (O(n)). Corregido: GestorUsuarios indexa los usuarios en la tabla hash (`buscarUsuario`) y la versión lineal se conserva como `buscarUsuarioLineal` solo para comparar.
- *C6.* Los gestores usaban la tabla hash de suma ASCII. Corregido: `estructuras/__init__.py` exporta como TablaHash la tabla djb2 de `Estructuras.py`; la de suma ASCII (`TablaHash.py`) queda como TablaHashASCII para comparar.
- *C7.* El contador de préstamos del recurso no se incrementaba al prestar. Corregido: el registro del préstamo llama a `recurso.registrarPrestamo()` y deshacer lo descuenta.

- *C8.* Los listados de M06 usaban bubble sort. Corregido: usan `mergeSort` de `estructuras/Algoritmos.py`.
- *C9.* Había carpetas con la numeración anterior de módulos. Corregido: las carpetas de `modulos` van de M01 a M08 como en este informe.
- *C10.* El método de gamificación para préstamos no se podía llamar. Corregido: `registrarPrestamoGami` quedó como método de la clase y el gestor de préstamos lo usa.
- *C11 (pendiente).* El caso exitoso de M04 en `test_pruebas.py` pide turno sin registrar la sede "Centro" y el caso cuello de botella usa códigos de usuario de cuatro dígitos (U0001), que el sistema no genera; por eso la prueba de M04 falla. Fue detectado por la integración continua.

Fase 3 (Diseño y análisis de algoritmos)

1. Tipo y volumen de los datos del caso

- *Usuarios:* códigos U001..., entre 500 y 2.000. Operación más frecuente: buscar por código.
- *Recursos:* códigos R0001..., 500 o más. Se buscan por código y se listan ordenados.
- *Préstamos:* códigos P0001..., miles por semestre. Se buscan por código y se filtran por estado, usuario o sede.
- *Solicitudes de turno:* decenas por recurso. Se encolan y desencolan.
- *Categorías:* decenas. Se consultan por código.
- *Sedes:* entre 3 y 10. Se valida que existan.

Todas las claves son códigos de texto generados en orden por un contador, por lo que son muy parecidos entre sí; esto afecta el diseño de la función hash. Los volúmenes van de cientos a pocos miles, suficientes para que la diferencia entre $O(n^2)$ y $O(n \log n)$ sea notable.

2. Justificación de los algoritmos de búsqueda y ordenamiento

2.1. Búsquedas

- *Tabla hash:* para buscar por código exacto, que es la operación más frecuente del mostrador. Su costo promedio no depende del número de registros: la tabla djb2 se mantuvo en 1,0 µs con 500 y con 10.000 códigos. Se usa en usuarios, recursos, préstamos, solicitudes y categorías.
- *Búsqueda binaria:* para ubicar un elemento en un listado ya ordenado. Es $O(\log n)$: con 10.000 elementos hace máximo 14 comparaciones (2,0 µs). Como los códigos se generan en orden, el vector de recursos ya está ordenado por código. Se implementó en versión iterativa y recursiva.

- *Búsqueda secuencial:* para filtros que no son la clave (activos, atrasados, por usuario, por sede). Es O(n), pero con miles de préstamos tarda menos de un milisegundo y no justifica un índice por filtro. No debe usarse para buscar por código; por eso los usuarios, que en la Entrega 1 se buscaban así, pasaron a la tabla hash (C5).

2.2. Ordenamientos

Se implementaron cuatro en `estructuras/Algoritmos.py` (`bubbleSort`, `insercion`, `mergeSort` y `quickSort`), todos con parámetros `clave` y `reverso`, devolviendo una lista nueva.

- *Merge sort (avanzado):* elegido para los listados del informe porque es estable y siempre O(n log n). La estabilidad permite que los recursos con la misma cantidad de préstamos conserven su orden previo. Con 2.000 recursos tarda 5,1 ms.
- *Quick sort (avanzado):* el más rápido en las mediciones (3,8 ms con 2.000 y 20,4 ms con 10.000). Se usa con claves únicas, donde no importa la estabilidad. El pivote central y la recursión sobre la partición menor evitan el peor caso con datos ordenados y limitan la pila a O(log n).
- *Inserción (básico):* para listas pequeñas o casi ordenadas. Con 2.000 recursos y 2 % fuera de lugar tarda 10,7 ms, frente a 131 ms con datos aleatorios.
- *Burbuja (básico):* solo como referencia. Es el más lento (326 ms con 2.000 recursos); por eso los listados de M06 pasaron a merge sort (C8).

3. Tabla de complejidad Big-O de todos los módulos

n = elementos de la colección del gestor, u = usuarios, s = sedes, q = personas en la fila de un recurso, h = altura del árbol, α = factor de carga de la tabla hash. "Prom." indica caso promedio.

3.1. Estructuras propias

Estructura	Operación	Complejidad	Dónde se usa
VectorDinamico	agregar	O(1) amortizado	M01, M02, M03, M08
	obtener(i)	O(1)	
	buscar, eliminarxIndice, recorrer	O(n)	
ListaSimple	agregarAlInicio, eliminarInicio	O(1)	M03 historial
	agregarAlFinal, eliminarFinal, buscar, recorrer	O(n)	
ListaDoble	agregar/eliminar en ambos extremos	O(1)	M03 historial navegable
	buscar, recorrer adelante/atrás	O(n)	

Pila	push / pop / peek	O(1) (push amortizado)	M03 deshacer
ColaFIFO (circular)	encolar / desencolar / frente	O(1) (encolar amortizado)	M04 fila por recurso
TablaHash (djb2)	insertar, buscar, eliminar	O(1 + α) prom., O(n) peor	M01, M02, M03, M04, M05
	obtenerTodasLasClaves	O(m + n)	
ArbolBinario (BST)	insertar	O(h): O(log n) prom., O(n) peor	M05
	buscar(criterio), recorridos, altura	O(n)	
Grafo (lista de adyacencia)	agregar_arista / BFS / DFS	O(V) / O(V + E) teórico	E3 (sin integrar)

El árbol de M05 recibe códigos C001, C002... en orden, por lo que sin balanceo se degenera en una lista (h = n); esto justifica el AVL de la Entrega 3. El BFS/DFS del grafo marca los visitados en una lista de Python, lo que en la práctica cuesta O(V·(V + E)); al integrarlo debe usarse un ConjuntoHash para lograr O(V + E).

3.2. Operaciones de cada módulo

Módulo	Operación	Complejidad	Comentario
M01	registrar	O(1) amortizado	Vector + índice hash + set de códigos
	buscarUsuario / buscarxCodigo	O(1 + α) prom.	Tabla hash djb2 (C5)
	buscarUsuarioLineal	O(n)	Versión de la Entrega 1, solo para comparar
	actualizar, darDeBaja, reactivar	O(1 + α) prom.	Dominados por la búsqueda
	listarTodos / listarActivos	O(n)	
	codigoExiste	O(1) prom.	set
M02	registrarLibro / registrarEquipo / registrarSala	O(1) amort.	Vector + índice hash
	buscarxCodigo, darDeBaja	O(1 + α) prom.	Tabla hash djb2 con redimensionamiento
	listarTodos, listarxTipo	O(n)	
	registrarSede	O(1) prom.	
	listarSedes	O(s)	

M03	registrarPrestamo	O(s)	Usuario y recurso por hash; valida la sede recorriendo la lista
	registrarDevolucion	O(1 + α) (+ asignación de turno)	Préstamo y usuario por hash
	deshacer un préstamo	O(n)	Reconstruye vector y listas sin ese préstamo
	deshacer una devolución	O(1 + α) prom.	
	buscarPrestamo	O(1 + α) prom.	Índice hash
	listarActivos, obtenerPrestamosAtrasados / Usuario / Sede	O(n)	Búsqueda secuencial con filtro
	historialAdelante / historialAtras	O(n)	Recorre la lista doble
	recursoPrestado	O(1 + α) prom.	Índice de activos por recurso
	atenderSiguienteTurno	O(k·s)	k = solicitudes saltadas por usuario inactivo
M04	encolar	O(1) amort.	Hash para hallar la cola + encolar
	solicitarTurno	O(s + q)	Validaciones + revisión de duplicados en la fila
	verSiguiente, obtenerSiguiente, obtenerCantidadEspera	O(1)	
	obtenerFila	O(q)	
	cancelarSolicitud, reinsertarAlFrente	O(q)	Reconstruyen la cola
	obtenerCantidad (todas las filas)	O(m + total)	Recorre la tabla de colas
M05	registrarCategoria	O(h): O(log n) prom., O(n) con códigos ordenados	Hash para repetidos + inserción en BST
	obtenerCategoria	O(1 + α) prom.	Tabla hash
M06	generarReporteUso	O(1)	
	generarReporteInventario	O(n)	Tres pasadas por tipo
	listarRecursosOrdenadosPrestamos / Nombre, listarUsuariosOrdenadosNombre	O(n log n)	mergeSort, estable (C8)
	mergeSort / quickSort	O(n log n) / O(n log n) prom., $O(n^2)$ peor	Merge usa O(n) de memoria extra
	insercion / bubbleSort	$O(n^2)$, O(n) si casi ordenado	
	busquedaBinaria / busquedaSecuencial	O(log n) / O(n)	

M07	obtenerDisponibilidad / incrementar / decrementar / actualizar	O(1) prom.	Mapeo a índice + acceso a la celda
	registrarRecurso	O(1) amort.; O(s·n) al crecer	Duplica columnas como el vector
	obtenerFila / obtenerColumna	O(n) / O(s)	
	mostrarMatriz	O(s·n)	
M08	registrarPrestamoHist / quitarPrestamoHist	O(1) amort. / O(H)	H préstamos del historial
	recomendarRecursos	O(U·H)	U usuarios distintos
	registrarCalificacion / obtenerPromedioxRecurso	O(1) amort. / O(C)	C calificaciones
	rankingRecursos / leaderboard	O(C + K log K) / O(G log G)	Usa sort de Python
	predecirDisponibilidad, agregarPuntos	O(d) / O(1)	d préstamos finalizados del recurso

4. Diseño de la función hash propia y análisis de colisiones

4.1. Versión inicial: suma de códigos ASCII

La tabla de `estructuras/TablaHash.py`, que usaban los gestores en la Entrega 1 y hoy se conserva como TablaHashASCII solo para comparar, calcula h(clave) = (suma de ord(c)) mod tamaño, con tamaño 100 en recursos y 211 en préstamos, turnos y categorías, encadenamiento con listas enlazadas y sin redimensionamiento. Con los códigos reales presenta dos problemas:

- La suma no depende del orden de los caracteres: R0019, R0091, R0109, R0190, R0901 y R0910 suman 284 y caen en la misma cubeta.
- El rango de sumas es muy estrecho: de R0001 a R0500 solo hay 22 sumas distintas, así que nunca se usan más de 22 cubetas, sin importar el tamaño de la tabla.

4.2. Diseño propio: djb2 con tamaño primo y redimensionamiento

La tabla de `estructuras/Estructuras.py`, que desde esta entrega usan todos los gestores, inicia con h = 5381 y por cada carácter calcula h = (h · 33 + ord(c)) truncado a 32 bits, implementado como `((h << 5) + h + ord(c)) & 0xFFFFFFFF`. El índice es h mod p, con p primo (211 al inicio), y cuando el factor de carga supera 0,75 la tabla crece al siguiente primo ≥ 2p y reubica los nodos.

- Multiplicar por 33 hace que cada carácter pese según su posición: R0019 y R0091 quedan en las cubetas 151 y 196.
- Truncar a 32 bits mantiene el valor acotado aunque las claves sean largas, ya que los enteros de Python no tienen límite de tamaño.
- El tamaño primo reparte mejor los códigos consecutivos que un tamaño como 100.

- El redimensionamiento con factor 0,75 mantiene las cadenas cortas con costo O(1) amortizado por inserción.
- El encadenamiento separado evita que la tabla se llene y simplifica la eliminación.

4.3. Análisis de colisiones sobre los datos reales

Se insertaron los códigos con el formato que genera el sistema y se midieron las cubetas ocupadas, las colisiones (claves − cubetas ocupadas), la cadena más larga y las comparaciones promedio de una búsqueda exitosa.

- *500 recursos (R0001–R0500, igual para U001–U500 y S0001–S0500):* con suma ASCII, tanto en tabla de 100 como de 211, se ocupan 22 cubetas, hay 478 colisiones, cadena máxima de 44 y 16,42 comparaciones. Con djb2 en 211 fijo: 168 cubetas, 332 colisiones, cadena de 6 y 2,32 comparaciones. Con djb2 y redimensionamiento (863 cubetas, factor 0,58): 421 cubetas, 79 colisiones, cadena de 3 y 1,16 comparaciones.
- *2.000 préstamos (P0001–P2000):* con suma ASCII en 211 se ocupan 28 cubetas, hay 1.972 colisiones, cadena de 150 y 55,5 comparaciones. Con djb2 en 211 fijo: 211 cubetas, 1.789 colisiones, cadena de 18 y 6,4 comparaciones. Con redimensionamiento (3.467 cubetas, factor 0,58): 1.538 cubetas, 462 colisiones, cadena de 2 y 1,23 comparaciones.
- *3.500 claves mezcladas (U, R, P y S):* con suma ASCII se usan 50 cubetas, cadena de 237 y 76,7 comparaciones. Con djb2 fijo: cadena de 27 y 9,6 comparaciones. Con redimensionamiento (6.947 cubetas, factor 0,50): 2.950 cubetas, cadena de 3 y 1,16 comparaciones.

Con la suma ASCII la tabla se comporta casi como una lista: buscar un préstamo entre 2.000 cuesta en promedio 55,5 comparaciones. djb2 con tamaño fijo reduce las colisiones pero no las evita con factores de carga altos; la combinación de djb2 con redimensionamiento mantiene el factor entre 0,5 y 0,75 y la búsqueda en 1,2 comparaciones. Como referencia, una distribución aleatoria de 500 claves en 211 cubetas ocuparía unas 191 ($211 \cdot (1 - e^{-500/211})$); djb2 ocupó 168, porque códigos consecutivos producen hashes consecutivos. Con la tabla redimensionada a 863 se esperaban 379 y djb2 ocupó 421. Por eso la función que quedó integrada en el sistema es djb2 con tamaño primo y redimensionamiento (C6).

5. Resultados de la medición empírica de tiempos en Python

5.1. Metodología de medición

Se escribió un script que importa las funciones del repositorio (`estructuras/Algoritmos.py`, las dos tablas hash, GestorUsuarios y GestorRecursos) y mide con `time.perf_counter()`. Cada prueba se repite de 3 a 7 veces y se reporta la mediana. Se verificó que los cuatro ordenamientos

producen el mismo resultado que el ordenamiento de Python. Entorno: Python 3.13.16 en Linux x86-64.

5.2. Ordenamientos

Tiempos al ordenar recursos por cantidad de préstamos con datos aleatorios (burbuja / inserción / merge sort / quick sort, en ms):

- n = 100: 1,34 / 0,56 / 0,29 / 0,16.
- n = 250: 8,10 / 3,44 / 0,85 / 0,55.
- n = 500: 24,19 / 7,43 / 1,02 / 0,67.
- n = 1.000: 78,33 / 35,21 / 2,33 / 1,51.
- n = 2.000: 326,24 / 131,21 / 5,10 / 3,76.
- n = 5.000 y 10.000 (solo avanzados): merge sort 16,01 y 34,02; quick sort 9,64 y 20,40.

Con datos casi ordenados (2 % fuera de lugar): n = 500: 16,57 / 0,58 / 0,86 / 0,52; n = 1.000: 104,15 / 2,79 / 2,01 / 1,38; n = 2.000: 331,02 / 10,67 / 4,74 / 2,55.

- Al duplicar n de 1.000 a 2.000, burbuja se multiplicó por 4,2 e inserción por 3,7 (comportamiento cuadrático); merge sort por 2,2 y quick sort por 2,5, cercano a lo esperado para n log n. De 5.000 a 10.000 ambos se multiplicaron por 2,1.
- Con 2.000 recursos, merge sort es 64 veces más rápido que burbuja.
- Quick sort fue entre 1,4 y 1,8 veces más rápido que merge sort; se prefiere merge sort en los listados con empates por su estabilidad.
- Con datos casi ordenados, inserción baja de 131 ms a 10,7 ms con 2.000 elementos, mientras burbuja casi no mejora.

5.3. Búsquedas

Microsegundos por consulta, promedio de 2.000 búsquedas (secuencial / binaria / hash suma ASCII / hash djb2):

- n = 500: 13,83 / 1,22 / 1,10 / 0,98.
- n = 1.000: 29,49 / 1,55 / 1,59 / 1,08.
- n = 5.000: 161,33 / 1,93 / 6,51 / 1,02.
- n = 10.000: 336,21 / 1,99 / 11,96 / 1,02.

La secuencial crece linealmente; la binaria pasa de 1,2 a 2,0 µs mientras n se multiplica por 20; la hash djb2 se mantiene en 1,0 µs. La hash de suma ASCII empata con djb2 en 500 claves, pero con 10.000 es 12 veces más lenta por las cadenas largas.

5.4. Búsqueda en los gestores del proyecto

Microsegundos por búsqueda por código con las clases del repositorio (usuario lineal de la Entrega 1 / usuario con hash / recurso con hash): 100 registros: 4,72

/ 0,78 / 0,88; 500: 23,86 / 0,79 / 0,91; 1.000: 49,16 / 0,74 / 0,90; 2.000: 98,27 / 0,85 / 0,88.

La búsqueda lineal de la Entrega 1 crece en proporción a n, mientras que con la tabla djb2 la búsqueda de usuarios y de recursos se mantiene por debajo de 1 µs en todos los tamaños; con 2.000 usuarios es más de 100 veces más rápida. Esto confirma las correcciones C5 y C6.

6. Conclusiones

La implementación mostró que algunas decisiones del primer informe, como la búsqueda por código como módulo independiente o la matriz sede × día, no correspondían con el uso real del sistema; reorganizarlo en 8 módulos simplificó las dependencias. Las mediciones confirmaron el análisis Big-O: para el volumen del caso, la búsqueda por código debe hacerse con tabla hash y los listados con merge sort, y una función hash mal diseñada puede volver O(n) una estructura que en teoría es O(1). Las correcciones C5 a C10 ya quedaron aplicadas en el código. Para la Entrega 3 quedan la red de sedes con grafos, el AVL de categorías, la persistencia en archivos y la corrección de la prueba de M04 (C11).